

Software Testing & Quality Assurance

Goals of Software Testing

Software Testing:

- Testing produces reliability and quality
- Quality leads to customer satisfaction

Immediate Goals:

- Bug Discovery
- Bug Prevention

Long-term Goals:

- Reliability
- Quality
- Customer satisfaction
- Risk Management

Post-Implementation Goals:

- Reduced maintenance cost
- Improved testing process

The major **objectives of Software testing are as follows:**

- Finding defects which may get created by the programmer while developing the software.
- Gaining confidence in and providing information about the level of quality.
- To prevent defects.
- To make sure that the end result meets the business and user requirements.
- To ensure that it satisfies the BRS that is Business Requirement Specification and SRS that is System Requirement Specifications.
- To gain the confidence of the customers by providing them a quality product.

Principles of Software Testing:

1. Testing Shows Presence of Defects:

Testing shows the presence of defects in the software. The goal of testing is to make the software fail. Sufficient testing reduces the presence of defects. In case testers are unable to find defects after repeated regression testing doesn't mean that the software is bug-free.

Testing talks about the presence of defects and don't talk about the absence of defects.

2. Exhaustive testing is not possible:

It is the process of testing the functionality of a software in all possible inputs (valid or invalid) and pre-conditions is known as exhaustive testing. Exhaustive testing is impossible means the software can never test at every test cases. It can test only some test cases and assume that software is correct and it will produce the correct output in every test cases. If the software will test every test cases then it will take more cost, effort, etc. and which is impractical.

Principles of Software Testing:

3. Early Testing:

To find the defect in the software, early test activity shall be started. The defect detected in early phases of SDLC will be very less expensive. For better performance of software, software testing will start at initial phase i.e. testing will perform at the requirement analysis phase.

Assume two scenarios, first one is you have identified an incorrect requirement in the requirement gathering phase and the second one is you have identified a bug in the fully developed functionality. It is cheaper to change the incorrect requirement compared to fixing the fully developed functionality which is not working as intended.

4. Defect Clustering:

Defect Clustering in software testing means that a small module or functionality contains most of the bugs or it has the most operational failures.

As per the Pareto Principle (80-20 Rule), 80% of issues come from 20% of modules and remaining 20% of issues from remaining 80% of modules. So we do emphasize testing on the 20% of modules where we face 80% of bugs.

Principles of Software Testing:

5. Pesticide paradox:

Repeating the same test cases again and again will not find new bugs. So it is necessary to review the test cases and add or update test cases to find new bugs.

6. Testing is context dependent:

Testing approach depends on context of software developed. Different types of software need to perform different types of testing. For example, The testing of the e-commerce site is different from the testing of the Android application.

7. Absence of errors fallacy:

If a built software is 99% bug-free but it does not follow the user requirement then it is unusable. It is not only necessary that software is 99% bug-free but it also mandatory to fulfill all the customer requirements.

Software Testing - Myths

Myth 1: **Testing is Too Expensive**

Reality – There is a saying, pay less for testing during software development or pay more for maintenance or correction later. Early testing saves both time and cost in many aspects, however reducing the cost without testing may result in improper design of a software application rendering the product useless.

Myth 2: **Testing is Time-Consuming**

Reality – During the SDLC phases, testing is never a time-consuming process. However diagnosing and fixing the errors identified during proper testing is a time-consuming but productive activity.

Myth 3: **Only Fully Developed Products are Tested**

Reality – No doubt, testing depends on the source code but reviewing requirements and developing test cases is independent from the developed code. However iterative or incremental approach as a development life cycle model may reduce the dependency of testing on the fully developed software.

Software Testing - Myths

Myth 4: Complete Testing is Possible

Reality – It becomes an issue when a client or tester thinks that complete testing is possible. It is possible that all paths have been tested by the team but occurrence of complete testing is never possible. There might be some scenarios that are never executed by the test team or the client during the software development life cycle and may be executed once the project has been deployed.

Myth 5: A Tested Software is Bug-Free

Reality – This is a very common myth that the clients, project managers, and the management team believes in. No one can claim with absolute certainty that a software application is 100% bug-free even if a tester with superb testing skills has tested the application.

Myth 6: Missed Defects are due to Testers

Reality – It is not a correct approach to blame testers for bugs that remain in the application even after testing has been performed. This myth relates to Time, Cost, and Requirements changing Constraints. However the test strategy may also result in bugs being missed by the testing team.

Software Testing - Myths

Myth 7: Testers are Responsible for Quality of Product

Reality – It is a very common misinterpretation that only testers or the testing team should be responsible for product quality. Testers' responsibilities include the identification of bugs to the stakeholders and then it is their decision whether they will fix the bug or release the software. Releasing the software at the time puts more pressure on the testers, as they will be blamed for any error.

Myth 8: Test Automation should be used wherever possible to Reduce Time

Reality – Yes, it is true that Test Automation reduces the testing time, but it is not possible to start test automation at any time during software development. Test automation should be started when the software has been manually tested and is stable to some extent. Moreover, test automation can never be used if requirements keep changing.

Software Testing - Myths

Myth 9: **Anyone can Test a Software Application**

Reality – People outside the IT industry think and even believe that anyone can test a software and testing is not a creative job. However testers know very well that this is a myth. Thinking alternative scenarios, try to crash a software with the intent to explore potential bugs is not possible for the person who developed it.

Myth 10: **A Tester's only Task is to Find Bugs**

Reality – Finding bugs in a software is the task of the testers, but at the same time, they are domain experts of the particular software. Developers are only responsible for the specific component or area that is assigned to them but testers understand the overall workings of the software, what the dependencies are, and the impacts of one module on another module.

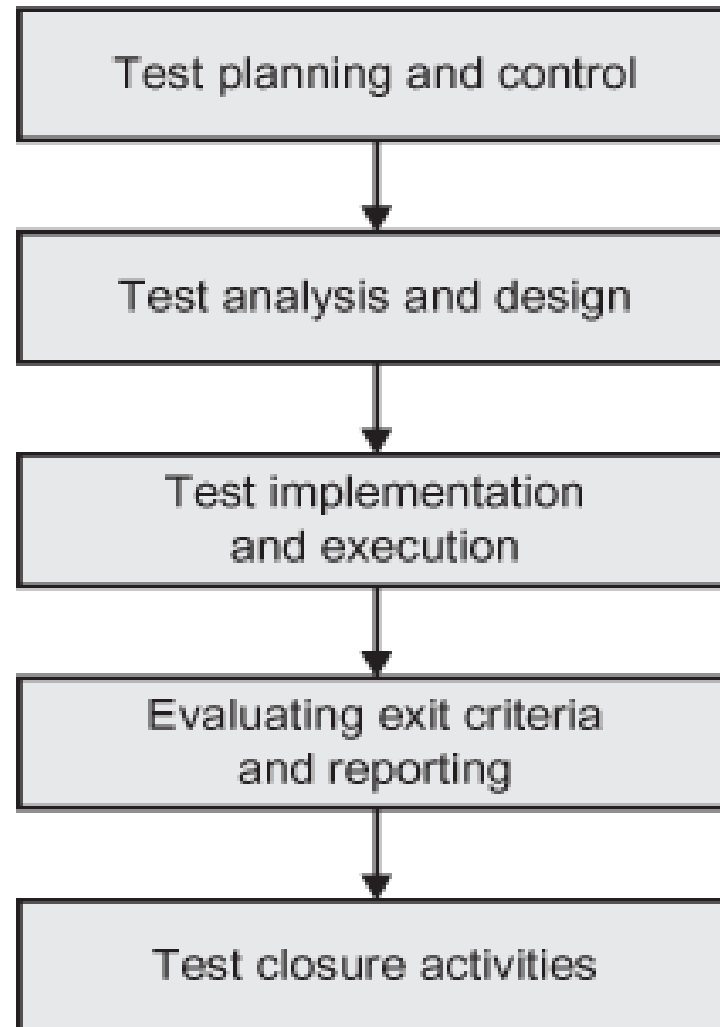
Test Process in Software Testing

Testing is a process rather than a single activity. Testing must be planned and it requires discipline to act upon it. The quality and effectiveness of software testing are primarily determined by the quality of the test processes used.

The activities of testing can be divided into the following basic steps:

- *Planning and Control*
- *Analysis and Design*
- *Implementation and Execution*
- *Evaluating exit criteria and Reporting*
- *Test Closure activities*

Test Process in Software Testing



Test Process in Software Testing

1) Planning and Control

Test Planning : Test planning involves producing a document that describes an overall approach and test objectives. It involves reviewing the test basis, identifying the test conditions based on analysis of test items, writing test cases and Designing the test environment. Completion or exit criteria must be specified so that we know when testing (at any stage) is complete.

Purpose

- ❑ *To determine the scope and risks and identify the objectives of testing.*
- ❑ *To determine the required test resources like people, test environments etc.*
- ❑ *To schedule test analysis and design tasks, test implementation, execution and evaluation.*

Control

This is the activity of comparing actual progress against the plan, and reporting the status, including deviations from the plan. It involves taking actions necessary to meet the mission and objectives of the project.

Test Process in Software Testing

2) Analysis and Design

Test analysis and Test Design has the following major tasks:

- ❑ *To review the test basis. The test basis is the information on which test cases are based, such as requirements, design specifications, product risk analysis, architecture and interfaces*
- ❑ *To identify test conditions*
- ❑ *To design the tests*
- ❑ *To design the test environment set-up and identify the required infrastructure and tools*

Test Process in Software Testing

3) Implementation and Execution

Test execution involves actually running the specified test on a computer system either manually or by using an automated test tool. It is a Fundamental Test Process in which actual work is done.

Test implementation has the following major task:

- ❑ *To develop and prioritize test cases by using techniques and create test data for those tests.*
- ❑ *To create test suites from the test cases for efficient test execution. Test suite is a collection of test cases that are used to test a software program*
- ❑ *To re-execute the tests that previously failed in order to confirm a fix.*
- ❑ *To log the outcome of the test execution. A test log is the status of the test case (pass/fail).*
- ❑ *To compare actual results with expected results.*

Test Process in Software Testing

4) Evaluating Exit criteria and Reporting

Evaluating exit criteria is a process defining when to stop testing. It depends on coverage of code, functionality or risk. Basically it also depends on business risk, cost and time and vary from project to project. Exit criteria come into picture, when:

- ❑ *Maximum test cases are executed with certain pass percentage*
- ❑ *Bug rate falls below certain level*
- ❑ *When we achieve the deadlines*

Evaluating exit criteria has the following major tasks:

- ❑ *To assess if more test are needed or if the exit criteria specified should be changed*
- ❑ *To write a test summary report for stakeholders*

Test Process in Software Testing

5) Test Closure activities:

Test closure activities are done when software is ready to be delivered. The testing can be closed for the other reasons also like:

- ❑ *When a project is cancelled*
- ❑ *When some target is achieved*
- ❑ *When a maintenance release or update is done*

Test closure activities have the following major tasks:

- ❑ *To check which planned deliverables are actually delivered and to ensure that all incident reports have been resolved*
- ❑ *To finalize and archive testware such as scripts, test environments, etc. for later reuse*
- ❑ *To handover the testware to the maintenance organization. They will give support to the software*
- ❑ *To evaluate how the testing went and learn lessons for future releases and projects*

Why Software Testing is Important?

Software Testing is Important because if there are any bugs or errors in the software, it can be identified early and can be solved before delivery of the software product. Properly tested software product ensures reliability, security and high performance which further results in time saving, cost effectiveness and customer satisfaction.

Testing is important because software bugs could be expensive or even dangerous. Software bugs can potentially cause monetary and human loss, and history is full of such examples.

- In April 2015, Bloomberg terminal in London crashed due to software glitch affected more than 300,000 traders on financial markets. It forced the government to postpone a 3bn pound debt sale.
- Nissan cars recalled over 1 million cars from the market due to software failure in the airbag sensory detectors. There has been reported two accident due to this software failure.

Why Software Testing is Important?

- Starbucks was forced to close about 60 percent of stores in the U.S and Canada due to software failure in its POS system. At one point, the store served coffee for free as they were unable to process the transaction.
- Some of Amazon's third-party retailers saw their product price is reduced to 1p due to a software glitch. They were left with heavy losses.
- Vulnerability in Windows 10. This bug enables users to escape from security sandboxes through a flaw in the win32k system.
- In 2015 fighter plane F-35 fell victim to a software bug, making it unable to detect targets correctly.

Why Software Testing is Important?

- China Airlines Airbus A300 crashed due to a software bug on April 26, 1994, killing 264 innocents live
- In 1985, Canada's Therac-25 radiation therapy machine malfunctioned due to software bug and delivered lethal radiation doses to patients, leaving 3 people dead and critically injuring 3 others.
- In April of 1999, a software bug caused the failure of a \$1.2 billion military satellite launch, the costliest accident in history
- In May of 1996, a software bug caused the bank accounts of 823 customers of a major U.S. bank to be credited with 920 million US dollars.

Defect/Bug Life Cycle in Software Testing

What is Defect Life Cycle?

Defect Life Cycle or Bug Life Cycle in software testing is the specific set of states that defect or bug goes through in its entire life. The purpose of Defect life cycle is to easily coordinate and communicate current status of defect which changes to various assignees and make the defect fixing process systematic and efficient.

Defect Status

Defect Status or Bug Status in defect life cycle is the present state from which the defect or a bug is currently undergoing. The goal of defect status is to precisely convey the current state or progress of a defect or bug in order to better track and understand the actual progress of the defect life cycle.

Defect/Bug Life Cycle in Software Testing

The number of states that a defect goes through varies from project to project. Below lifecycle diagram, covers all possible states:

New: When a new defect is logged and posted for the first time. It is assigned a status as NEW.

Assigned: Once the bug is posted by the tester, the lead of the tester approves the bug and assigns the bug to the developer team.

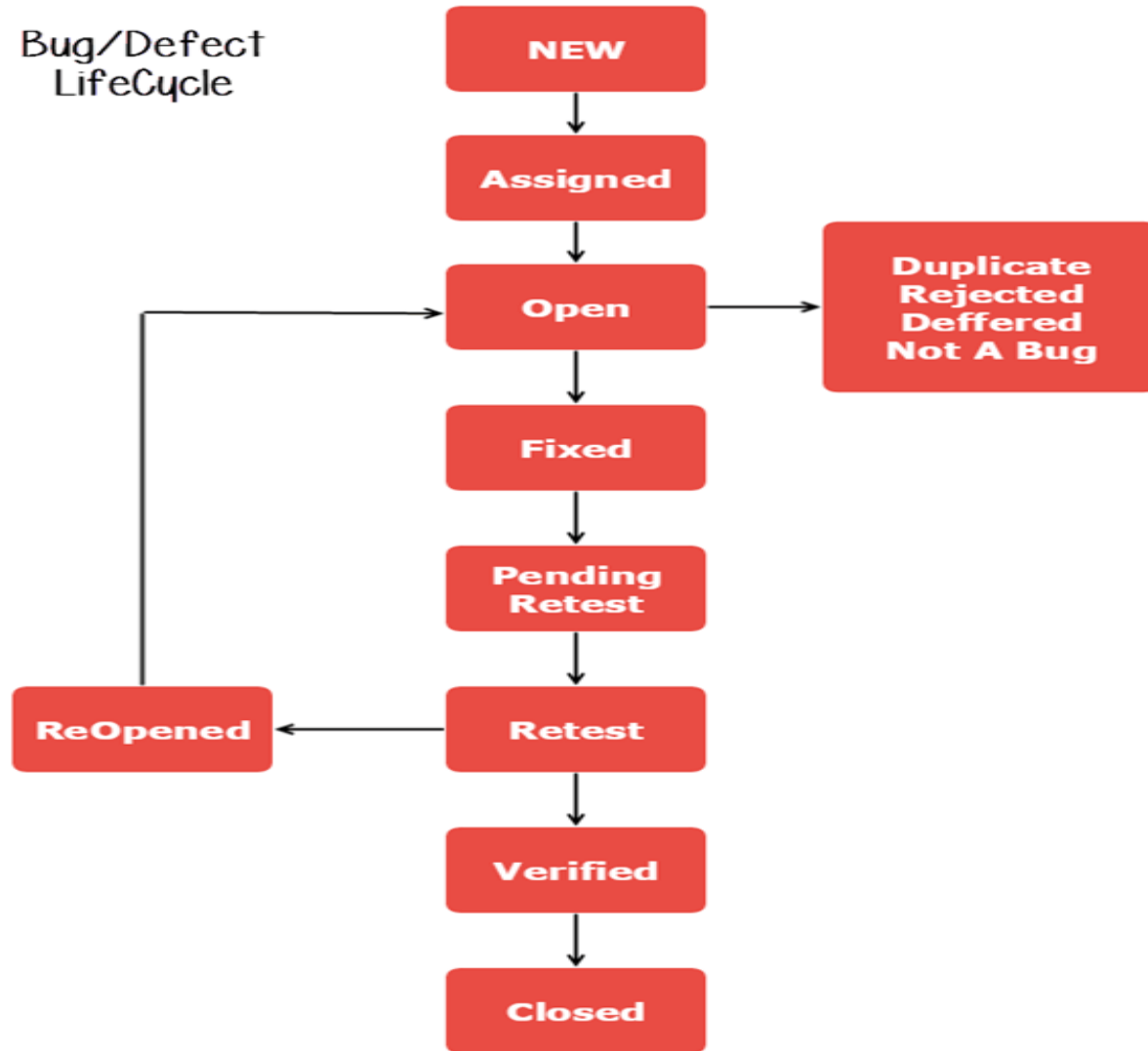
Open: The developer starts analyzing and works on the defect fix.

Fixed: When a developer makes a necessary code change and verifies the change, he or she can make bug status as "Fixed."

Pending Retest: After fixing the defect, the developer assigns the defect to the tester for retesting the defect at their end, and till the tester works on retesting the defect, the state of the defect remains in 'Pending Retest'.

Retest: Tester does the retesting of the code at this stage to check whether the defect is fixed by the developer or not and changes the status to "Re-test."

Defect/Bug Life Cycle in Software Testing



Defect/Bug Life Cycle in Software Testing

Verified: The tester re-tests the bug after it got fixed by the developer. If there is no bug detected in the software, then the bug is fixed and the status assigned is "verified."

Reopen: If the bug persists even after the developer has fixed the bug, the tester changes the status to "reopened". Once again the bug goes through the life cycle.

Closed: If the bug is no longer exists then tester assigns the status "Closed."

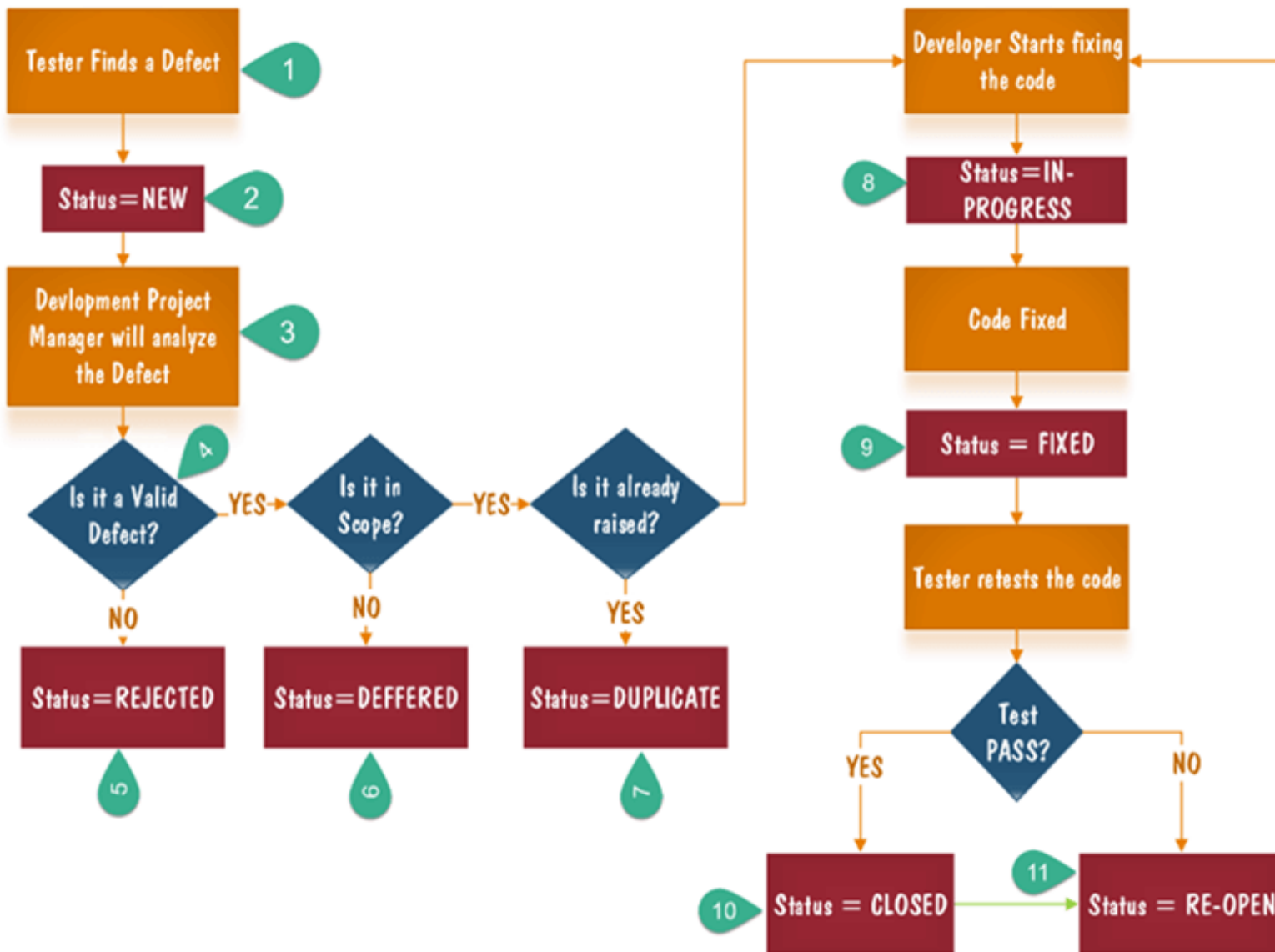
Duplicate: If the developer finds the defect as same as any other defect or if the concept of the defect matches any other defect then the status of the defect is changed to 'Duplicate' by the developer.

Rejected: If the developer feels the defect is not a genuine defect then it changes the defect to "rejected."

Deferred: If the present bug is not of a prime priority and if it is expected to get fixed in the next release, then status "Deferred" is assigned to such bugs.

Not a bug: If it does not affect the functionality of the application then the status assigned to a bug is "Not a bug".

Defect Life Cycle Explained



Defect Life Cycle Explained

1. Tester finds the defect
2. Status assigned to defect- New
3. A defect is forwarded to Project Manager for analyze
4. Project Manager decides whether a defect is valid
5. Here the defect is not valid- a status is given "Rejected."
6. So, project manager assigns a status rejected. If the defect is not rejected then the next step is to check whether it is in scope. Suppose we have another function- email functionality for the same application, and you find a problem with that. But it is not a part of the current release when such defects are assigned as a postponed or deferred status.

Defect Life Cycle Explained

7. Next, the manager verifies whether a similar defect was raised earlier. If yes defect is assigned a status duplicate.
8. If no the defect is assigned to the developer who starts fixing the code. During this stage, the defect is assigned a status in- progress.
9. Once the code is fixed. A defect is assigned a status fixed
10. Next, the tester will re-test the code. In case, the Test Case passes the defect is closed. If the test cases fail again, the defect is re-opened and assigned to the developer.
11. Consider a situation where during the 1st release of Flight Reservation a defect was found in Fax order that was fixed and assigned a status closed. During the second upgrade release the same defect again re-surfaced. In such cases, a closed defect will be re-opened.

Testing and Debugging

Testing:

Testing is the process of verifying and validating that a software or application is bug free, meets the technical requirements as guided by its design and development and meets the user requirements effectively and efficiently with handling all the exceptional and boundary cases.

Debugging:

Debugging is the process of fixing a bug in the software. It can be defined as the identifying, analyzing and removing errors. This activity begins after the software fails to execute properly and concludes by solving the problem and successfully testing the software. It is considered to be an extremely complex and tedious task because errors need to be resolved at all stages of debugging.

Differences between Testing and Debugging

TESTING	DEBUGGING
Testing is the process to find bugs and errors.	Debugging is the process to correct the bugs found during testing.
It is the process to identify the failure of implemented code.	It is the process to give the absolution to code failure.
Testing is the display of errors.	Debugging is a deductive process.
Testing is done by the tester.	Debugging is done by either programmer or developer.
There is no need of design knowledge in the testing process.	Debugging can't be done without proper design knowledge.
Testing can be done by insider as well as outsider.	Debugging is done only by insider. Outsider can't do debugging.

Differences between Testing and Debugging

TESTING	DEBUGGING
Testing can be manual or automated.	Debugging is always manual. Debugging can't be automated.
It is based on different testing levels i.e. unit testing, integration testing, system testing etc.	Debugging is based on different types of bugs.
Testing is a stage of software development life cycle (SDLC).	Debugging is not an aspect of software development life cycle, it occurs as a consequence of testing.
Testing is composed of validation and verification of software.	While debugging process seeks to match symptom with cause, by that it leads to the error correction.
Testing is initiated after the code is written.	Debugging commences with the execution of a test case.

What is Software Testing Life Cycle (STLC)?

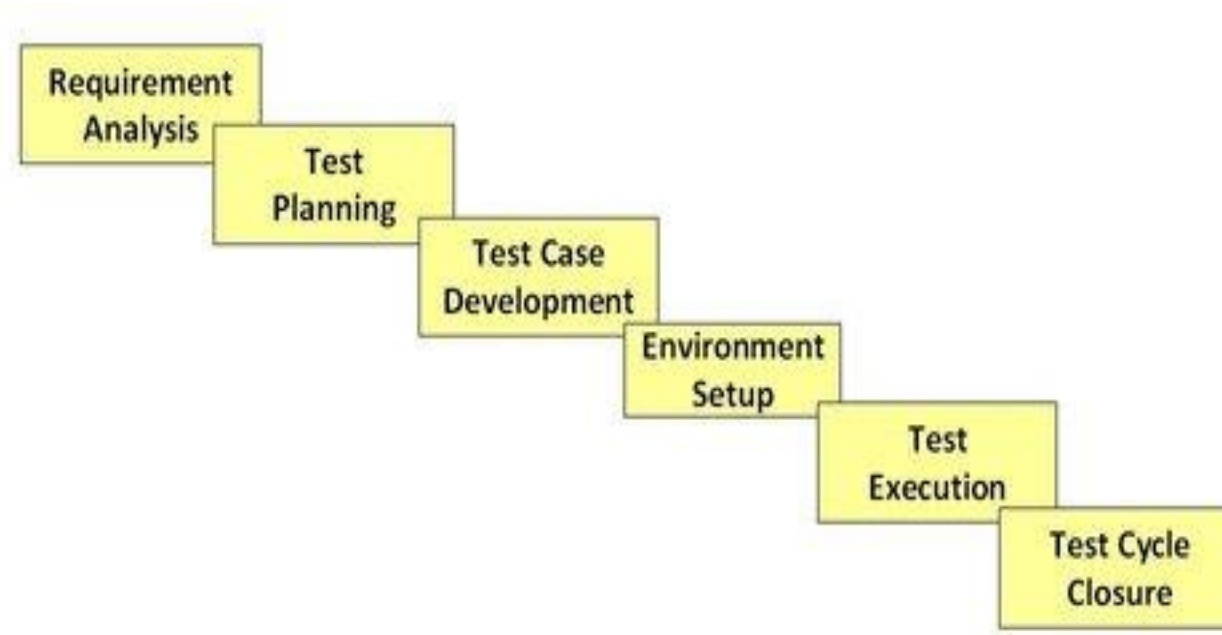
Software Testing Life Cycle (STLC) is a sequence of specific activities conducted during the testing process to ensure software quality goals are met. STLC involves both verification and validation activities. Contrary to popular belief, Software Testing is not just a single/isolate activity, i.e. testing. It consists of a series of activities carried out methodologically to help certify your software product. STLC stands for Software Testing Life Cycle.

STLC Phases

There are following six major phases in every Software Testing Life Cycle Model (STLC Model):

- Requirement Analysis
- Test Planning
- Test case development
- Test Environment setup
- Test Execution
- Test Cycle closure

What is Software Testing Life Cycle (STLC)?



STLC Model Phases

Each of these stages has a definite Entry and Exit criteria, Activities & Deliverables associated with it.

What is Software Testing Life Cycle (STLC)?

What is Entry and Exit Criteria in STLC?

Entry Criteria: Entry Criteria gives the prerequisite items that must be completed before testing can begin.

Exit Criteria: Exit Criteria defines the items that must be completed before testing can be concluded.

You have Entry and Exit Criteria for all levels in the Software Testing Life Cycle (STLC).

In an Ideal world, you will not enter the next stage until the exit criteria for the previous stage is met. But practically this is not always possible. So for this tutorial, we will focus on activities and deliverables for the different stages in STLC life cycle. Let's look into them in detail.

What is Software Testing Life Cycle (STLC)?

Requirement Phase Testing

Requirement Phase Testing also known as Requirement Analysis in which test team studies the requirements from a testing point of view to identify testable requirements and the QA team may interact with various stakeholders to understand requirements in detail. Requirements could be either functional or non-functional. Automation feasibility for the testing project is also done in this stage.

Activities in Requirement Phase Testing

- Identify types of tests to be performed.
- Gather details about testing priorities and focus.
- Prepare Requirement Traceability Matrix (RTM).
- Identify test environment details where testing is supposed to be carried out.
- Automation feasibility analysis (if required).

Deliverables of Requirement Phase Testing

- RTM
- Automation feasibility report. (if applicable)

What is Software Testing Life Cycle (STLC)?

Test Planning in STLC

Test Planning in STLC is a phase in which a Senior QA manager determines the test plan strategy along with efforts and cost estimates for the project. Moreover, the resources, test environment, test limitations and the testing schedule are also determined. The Test Plan gets prepared and finalized in the same phase.

Test Planning Activities

- Preparation of test plan/strategy document for various types of testing
- Test tool selection
- Test effort estimation
- Resource planning and determining roles and responsibilities.
- Training requirement

Deliverables of Test Planning

- Test plan /strategy document.
- Effort estimation document.

What is Software Testing Life Cycle (STLC)?

Test Case Development Phase

The **Test Case Development Phase** involves the creation, verification and rework of test cases & test scripts after the test plan is ready. Initially, the Test data is identified then created and reviewed and then reworked based on the preconditions. Then the QA team starts the development process of test cases for individual units.

Test Case Development Activities

- Create test cases, automation scripts (if applicable)
- Review and baseline test cases and scripts
- Create test data (If Test Environment is available)

Deliverables of Test Case Development

- Test cases/scripts
- Test data

What is Software Testing Life Cycle (STLC)?

Test Environment Setup

Test Environment Setup decides the software and hardware conditions under which a work product is tested. It is one of the critical aspects of the testing process and can be done in parallel with the Test Case Development Phase. Test team may not be involved in this activity if the development team provides the test environment. The test team is required to do a readiness check (smoke testing) of the given environment.

Test Environment Setup Activities

- Understand the required architecture, environment set-up and prepare hardware and software requirement list for the Test Environment.
- Setup test Environment and test data
- Perform smoke test on the build

Deliverables of Test Environment Setup

- Environment ready with test data set up
- Smoke Test Results.

What is Software Testing Life Cycle (STLC)?

Test Execution Phase

Test Execution Phase is carried out by the testers in which testing of the software build is done based on test plans and test cases prepared. The process consists of test script execution, test script maintenance and bug reporting. If bugs are reported then it is reverted back to development team for correction and retesting will be performed.

Test Execution Activities

- Execute tests as per plan
- Document test results, and log defects for failed cases
- Map defects to test cases in RTM
- Retest the Defect fixes
- Track the defects to closure

Deliverables of Test Execution

- Completed RTM with the execution status
- Test cases updated with results
- Defect reports

What is Software Testing Life Cycle (STLC)?

Test Cycle Closure

Test Cycle Closure phase is completion of test execution which involves several activities like test completion reporting, collection of test completion matrices and test results. Testing team members meet, discuss and analyze testing artifacts to identify strategies that have to be implemented in future, taking lessons from current test cycle. The idea is to remove process bottlenecks for future test cycles.

Test Cycle Closure Activities

- Evaluate cycle completion criteria based on Time, Test coverage, Cost, Software, Critical Business Objectives, Quality
- Prepare test metrics based on the above parameters.
- Document the learning out of the project
- Prepare Test closure report
- Qualitative and quantitative reporting of quality of the work product to the customer.
- Test result analysis to find out the defect distribution by type and severity.

Deliverables of Test Cycle Closure

- Test Closure report
- Test metrics

What are the Types of Software Testing Models?

Models in SDLC

Selecting proper testing methodologies incorporate with choosing a proper model in SDLC.

The models include:

- ✓ Waterfall model
- ✓ V model
- ✓ Agile model
- ✓ Spiral model
- ✓ RAD

Let's have closer look at each Software Development Methodologies with a brief explanation.

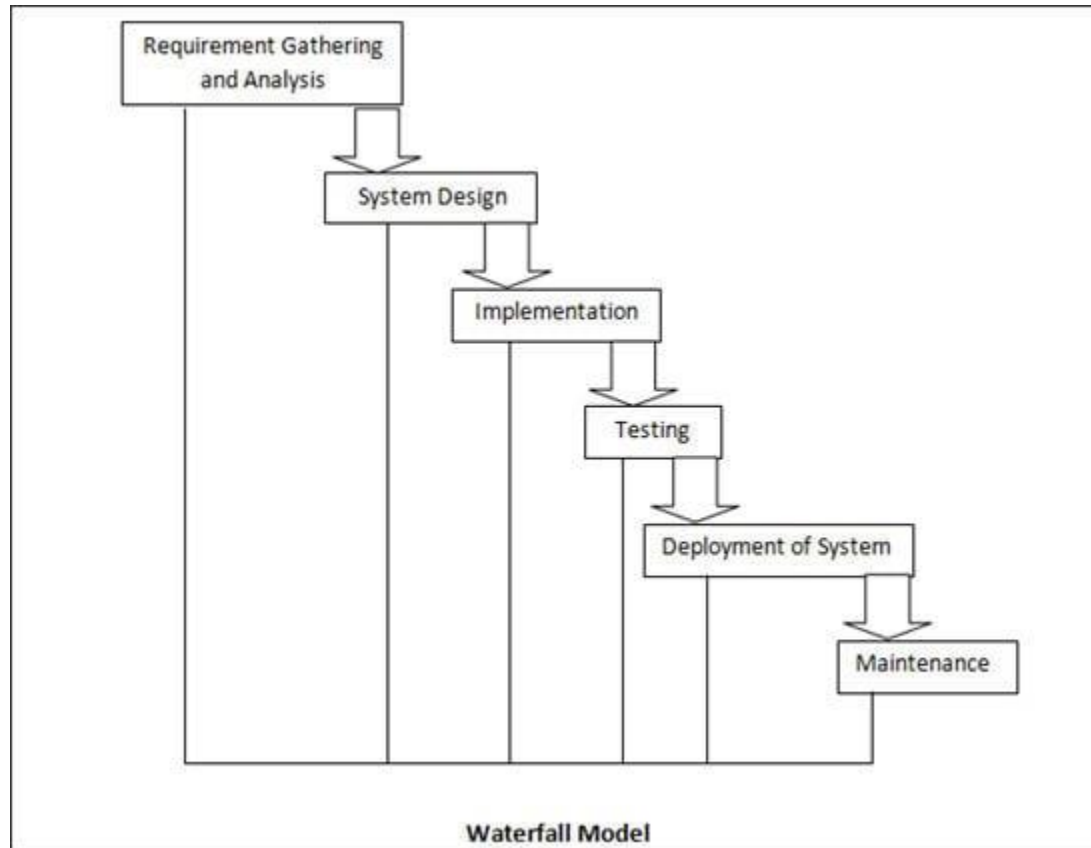
Waterfall Model

Waterfall model is the basic life cycle model which was developed by Winston Royce in 1970. This model represents multiple stages or processes in a sequential manner that flows progressively downward.

This approach is useful when requirements are well known, technology is understood and the resources with required expertise are available.

Waterfall Model

Waterfall model is defined by the following stages:



Waterfall Model

- **Requirement Gathering and Analysis:** Capture and analyze all the requirements and make sure whether they are testable or not.
- **System Design:** Create and document design based on requirement analysis. Define the hardware and software requirements.
- **Implementation:** Create robust code for components as per the design and integrate them.
- **System Testing:** Integrated components form a whole system, this phase is performed to ensure whether the system is working as per the requirements, tracking and reporting the testing progress.
- **System Deployment:** Make sure if system is stable with zero bugs, all test criteria had been met, ensure Environment Setup etc.
- **System Maintenance:** Makes sure if the application is working efficiently as per the requirement with the suitable environment. In case a defect is found then that should be fixed and deployed (updated) in the environment.

Waterfall Model

Advantages of Waterfall Model:

- ❖ Simple and easy to understand.
- ❖ Easy to manage as each phase has its own specific deliverables.
- ❖ Overlapping of stages is avoided.
- ❖ Good for small projects.

Disadvantages of Waterfall Model:

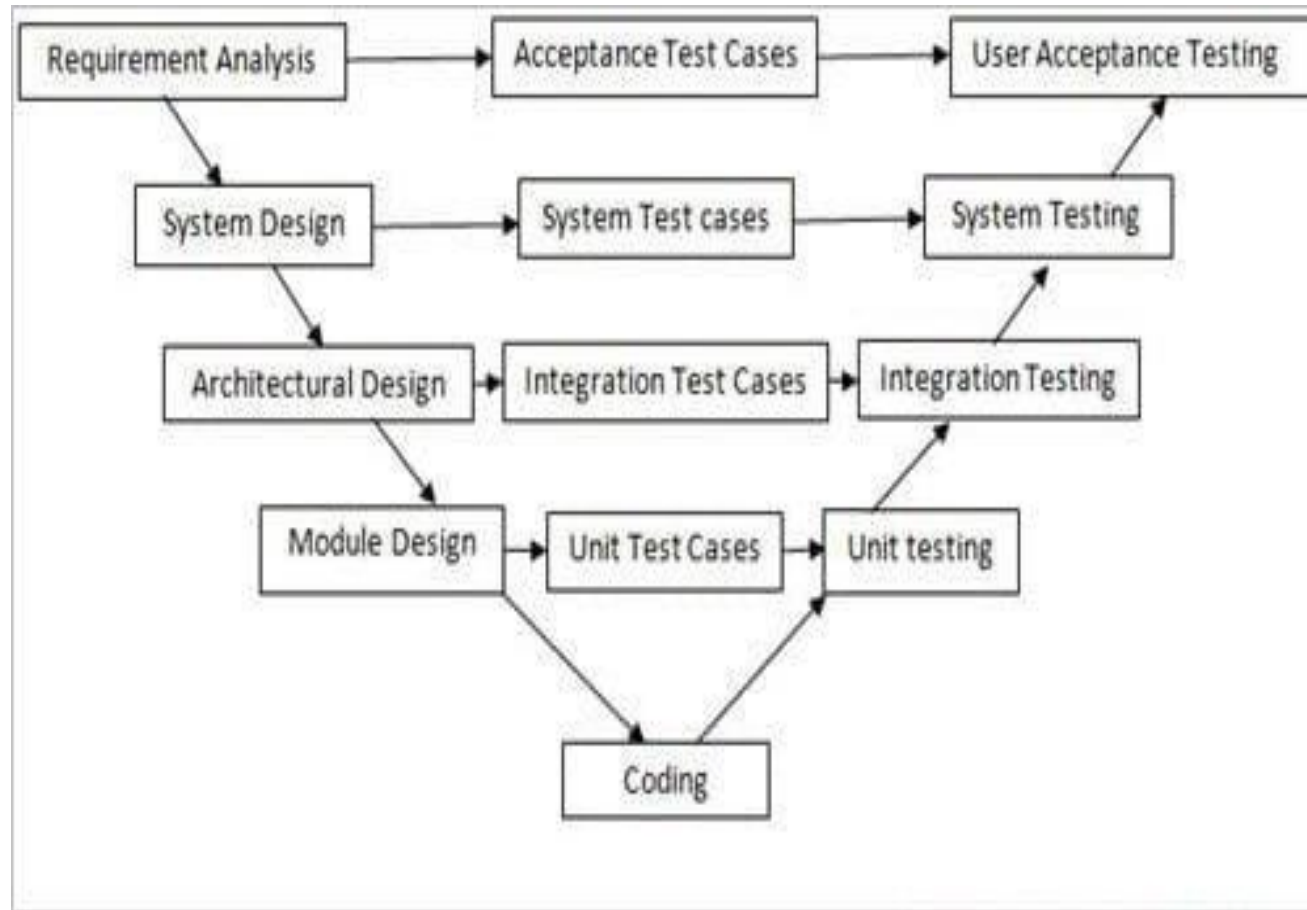
- ❖ Increase in the amount of risk and uncertainty.
- ❖ Once entered into Testing phase, cannot change anything in the previous stages e.g Design and Coding etc.
- ❖ Not good for complex and large projects.
- ❖ Not suitable where the requirements keep changing.

V Model

V Model is an *extension of Waterfall Model* where the process execution takes place in a sequential style in V-Shape and is also known as Verification and Validation Model. In this approach, there exists a directly associated testing phase in every single phase of the development cycle.

It has been proven beneficial and cost-efficient than the waterfall model as the testing is performed at each development phase rather than at the end of the development cycle.

V Model



V Model

V Model is classified into 3 Phases.

- Verification Phase
- Coding Phase
- Validation Phase

a) Verification Phase:

- ❖ **Business Requirement Analysis:** Communicate with the customer to understand their expectations and requirements.
- ❖ **System Design:** Design complete system and its components along with the hardware and software requirements.
- ❖ **Architectural Design:** In this phase architectural specifications are captured. This also known as high-level Design.
- ❖ **Module Design:** This is also known as Low-Level Design, Detailed internal design for all the specified system modules.

V Model

b) Coding Phase:

This phase contains actual coding phase in the development lifecycle. Programming languages should be chosen based on the system and architectural design specified in the previous phase technology platform. Coding is performed according to the standards and guidelines that are pre-defined.

c) Validation Phase:

- **Unit Testing:** Performed on an individual module to eliminate the bugs at the early stage.
- **Integration Testing:** Performed to test the communication between different modules in the system.
- **System Testing:** System Testing is performed on a system as a whole.
- **Acceptance Testing:** This is associated with the business requirements. It is performed in a user environment from the user's point of view.

V Model

Advantages of V model

- Simple, easy to use and understand.
- Overlapping is avoided as phases are executed one at a time.
- Easy to manage and suitable for small projects.

Disadvantages of V Model are more or less similar to the disadvantages of Waterfall model.

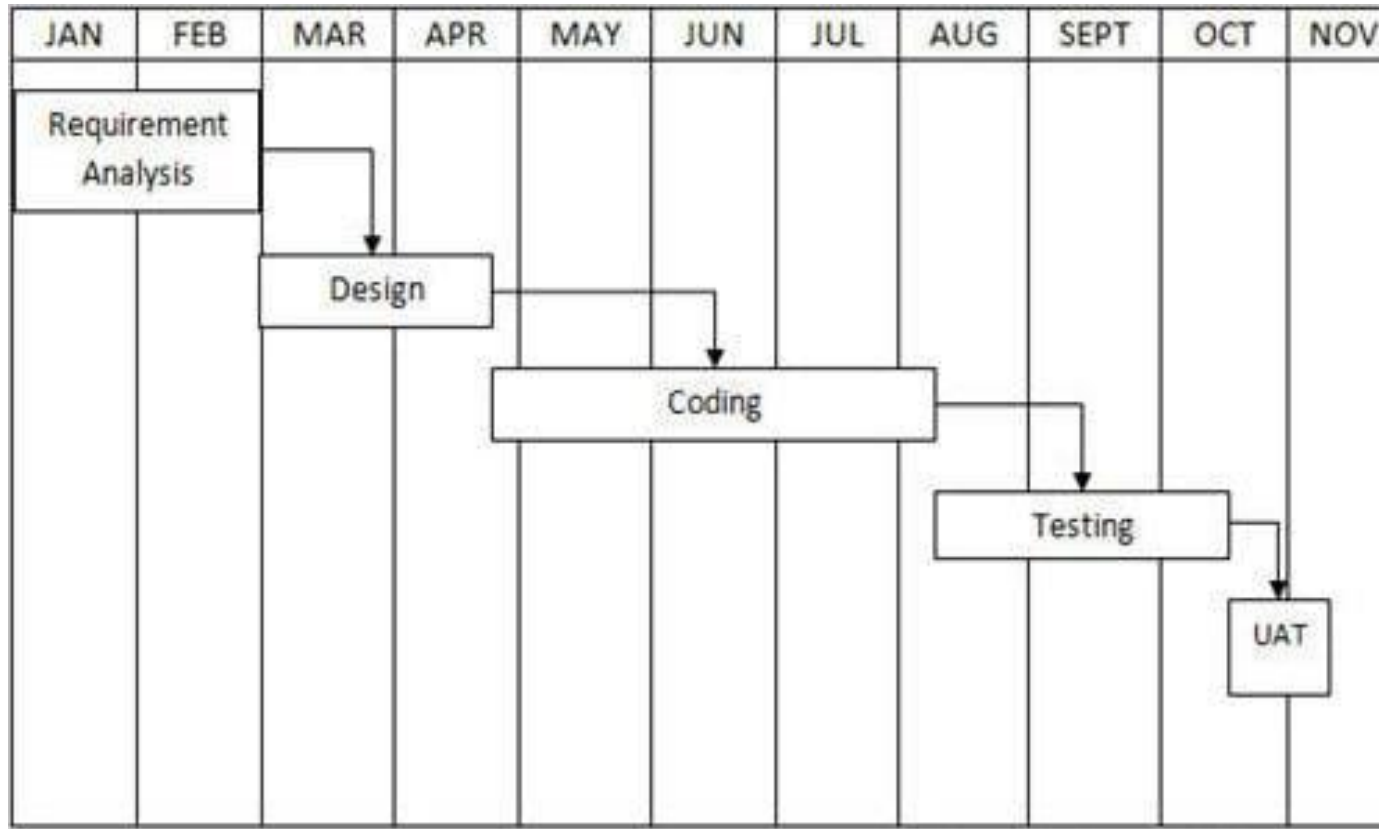
Agile Model

Agile Model shows an iterative and incremental approach. This approach breaks the product into small incremental units to provide iterations. Then each iteration involves steps like Planning, Requirement Analysis, Design, Coding, Unit Testing, Acceptance Testing etc.

This approach also allows continuous interaction with the customer for their feedback and corrections in the requirements at regular intervals.

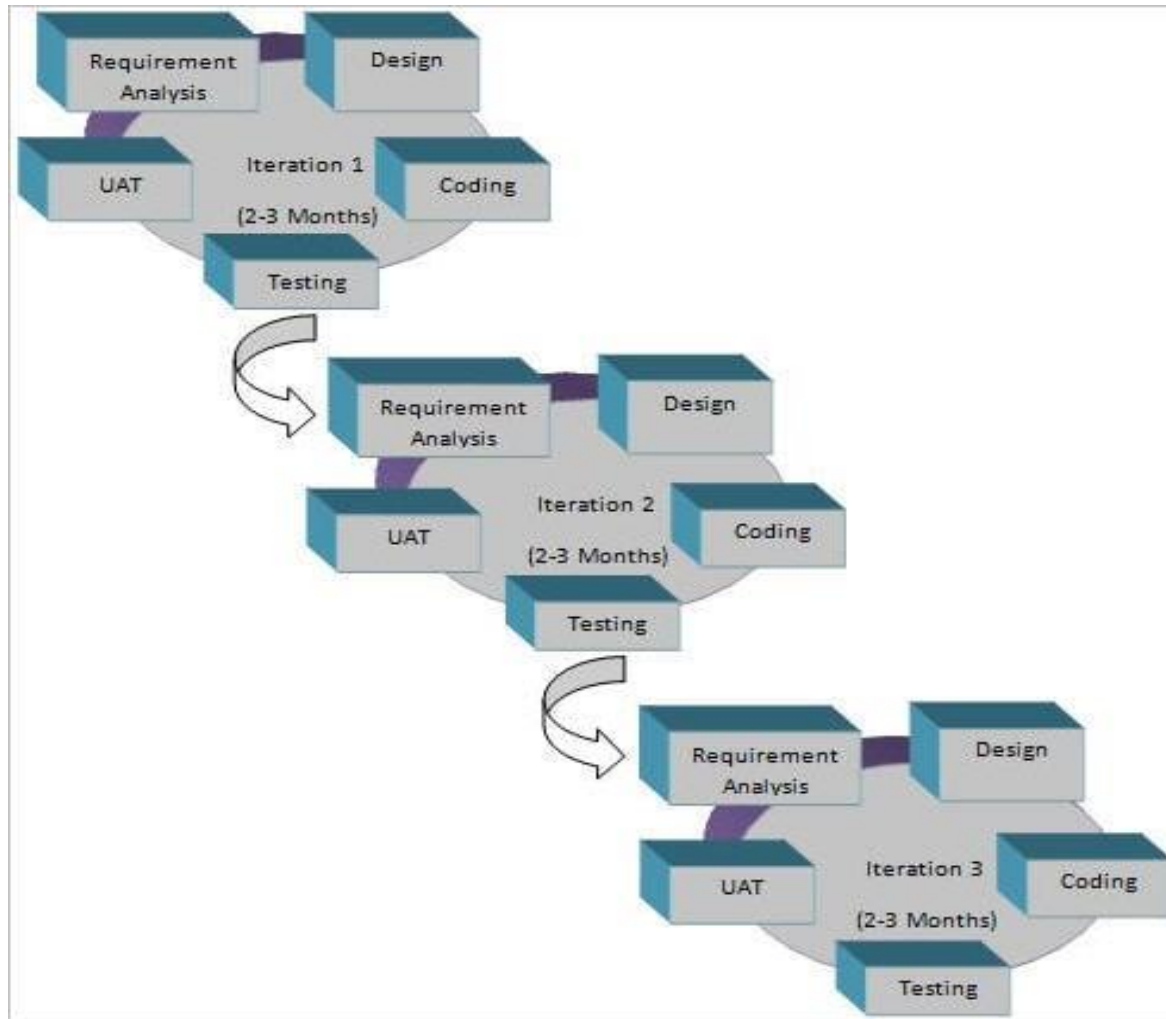
Agile Model

Following diagram will help you to understand Agile Model approach more precisely:



Agile Model

Following image will show the iteration cycle in Agile Model:



Agile Model

Advantages of Agile model:

- ❑ A realistic approach to software development.
- ❑ Promotes teamwork.
- ❑ Eliminates mismatch between requirements and test cases.
- ❑ Rapid and requires minimum amount of resources.
- ❑ Suitable for large and long-term projects.
- ❑ Good for changing requirements.
- ❑ Easy to manage.

Disadvantages of Agile model:

- ❑ Not suitable for complex projects.
- ❑ Requires heavy amount of interaction with the customer which may cause delay.
- ❑ Misguidance of requirements may cause the incorrect development of the software product.
- ❑ Increased maintainability risk.
- ❑ Handover to another team may be quite challenging.

Spiral Model

The spiral model incorporates iterative development approach along with the systematic approach of the waterfall model. It is similar to the incremental model and emphasis on Risk Analysis.

Spiral Model has four stages:

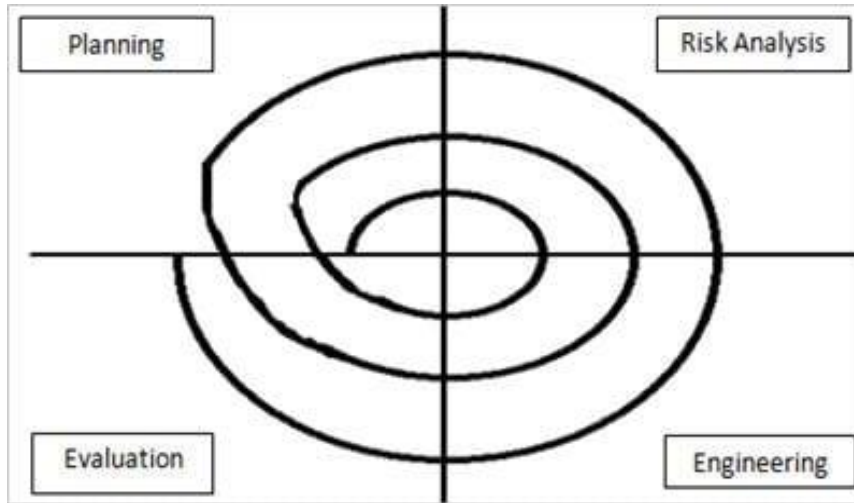
- Planning Phase
- Risk Analysis
- Engineering Phase
- Evaluation Phase

Spiral Model

- 1) Planning Phase:** In this phase, the requirements are gathered and reviewed to finalize the test case.
- 2) Risk Analysis:** This stage includes identifying, monitoring and estimating management risks. Requirements are analyzed to identify the risks using techniques like brainstorming, walkthrough etc.
- 3) Engineering Phase:** In this phase, the software is developed and tested at the end.
- 4) Evaluation Phase:** This is the last stage where a customer evaluates the output of a project and gives their feedback for either next spiral or approval.

Spiral Model

Pictorial representation of Spiral Model:



When to use Spiral model:

- For high-risk projects.
- When the requirements are complex.
- If a project is large.
- Have sufficient amount of time for getting user's feedback for the next spiral.
- Requires significant changes due to research and exploration.
- Users are not sure of their needs.

Spiral Model

Advantages of Spiral Model:

- ✓ Avoidance of risk as it involves a high amount of risk analysis.
- ✓ Rapid Development.
- ✓ Changes in requirements are accommodated easily.
- ✓ Requirements can be acquired more accurately.

Disadvantages of Spiral model:

- ✓ Complex management.
- ✓ Not suitable for small projects.
- ✓ May involve no. of spirals(indefinite).
- ✓ Costly.
- ✓ Requires high amount of risk analysis and expertise for their project's success.

RAD Model

Rapid Application Development (RAD) is a type of incremental model. In this approach, components are developed in parallel.

This is a rapid approach and it can give a fast product to the customer to provide feedback.

Phases in RAD are as follows:

Business Modeling: Identifies vital information and its flow between various business channels.

Data Modeling: Information gathered in the previous stage is used to define data objects required for the business.

Process modeling: Data objects are converted to get business objective and flow of information.

Application Generation: In this phase, automation tools are used to convert the process model into actual code.

Testing and Turnover: Tests all the components of a system, hence overall testing time is reduced.

RAD Model

Advantages of RAD Model:

- ❖ Progress can be measured.
- ❖ Reduces development time.
- ❖ Increased reusability.
- ❖ Quick initial reviews.
- ❖ Enhances customer feedback.

Disadvantages of RAD Model:

- ❖ Requires high skilled resources.
- ❖ High-cost estimation.
- ❖ Not applicable for cheaper projects.
- ❖ High dependency on modeling skills.
- ❖ Only a modularized system can be built using RAD.

Test Harness

A **test harness** or automated test framework is a collection of software and test data configured to test a program unit by running it under varying conditions and monitoring its behavior and outputs.

Defining Test Harness

It is a collection of stubs, drivers, and at the same time, other supporting tools that are required to execute tests. It performs tests by using a test library and generates a report. In addition to this, it contains all the information needed to compile and run a test. For instance, test cases, target deployment port (TDP), source file under test, stubs, etc.

Test Harness

STUBS & DRIVERS

Stubs and Drivers are two elements used in the software testing process, which act as a temporary replacement or substitute to the undeveloped or missing module. These are an integral part of the software testing process as well as general software development, as they are immensely useful in getting expected results.

Objectives of Test Harness:

- Automate the testing process.
- Helps in test case optimization by selecting specific tests to run
- Execute test suites of test cases.
- Organizes a run time environment
- Generate associated test reports.
- Analyzes test results.

Test Harness

Features of Test Harness:

- Executes test suit in the framework
- Assists measure the code coverage at the code level.
- Enters test data to an application under test
- Captures the output from the software under test
- Offers flexibility and support for debugging
- Records test results

Test Harness

Why use Test Harness?

- Automate the testing process
- Execute test suites of test cases
- Generate associated test reports
- Support for debugging
- To record the test results for each one of the tests
- Helps the developers to measure code coverage at a code level
- Increase the productivity of the system through automation
- Enhance the quality of software components and application
- To handle the complex condition that testers are finding difficult to simulate

Test Harness

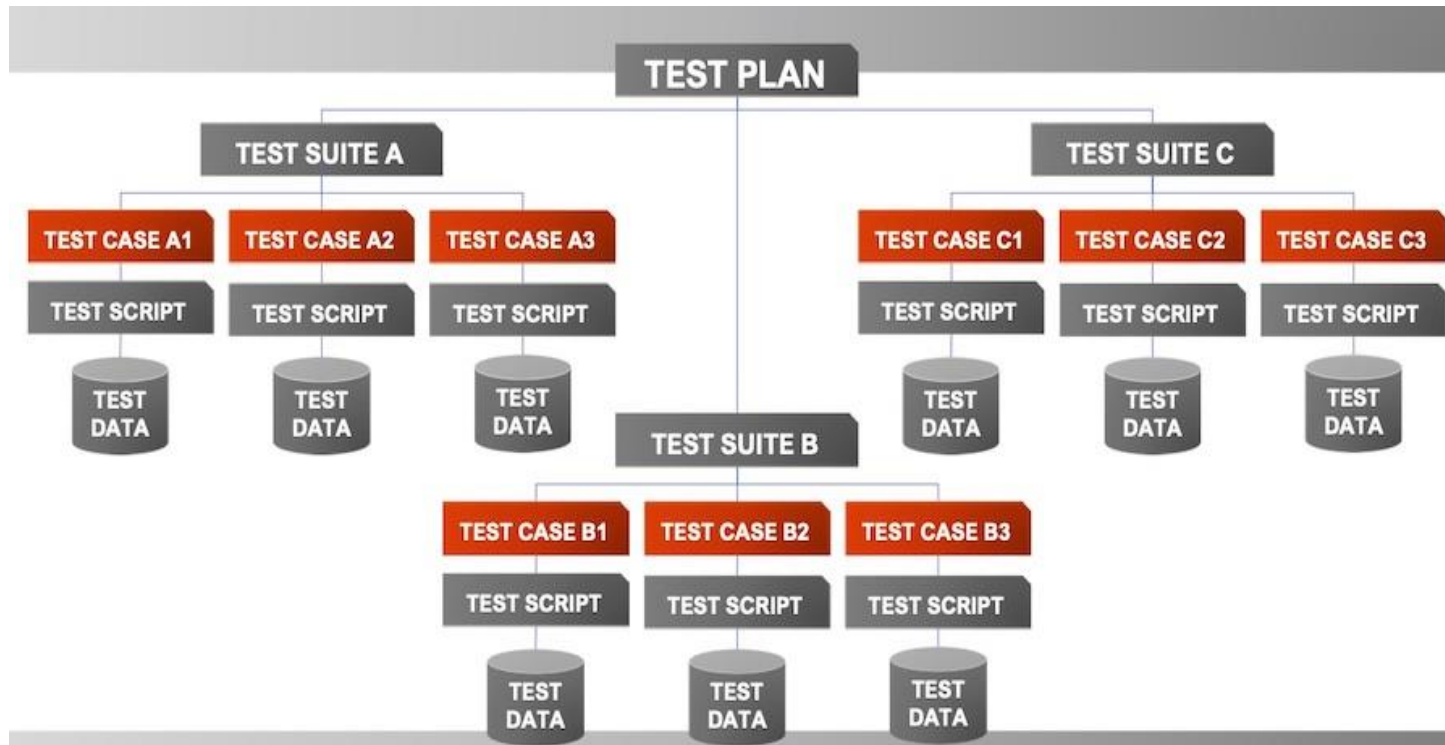
Advantages of Test Harness:

- With more effective automation, increases in productivity.
- Improved software quality.
- Automates the testing process.
- Supports debugging.
- Supports scheduling of tests.
- Good for complex conditions.
- Automated test reports generation.
- Records test results.

Disadvantages of Test Harness:

- Testers using test harness tools are expected to have knowledge of programming languages. For instance, JAVA, .NET, Groovy, Ruby, Python, etc.
- It could be an asset to any organization. At the same time, it incurs a cost for writing automates test suites or test cases through skilled developers.

Test Case



A **TEST CASE** is a documented set of preconditions (prerequisites), procedures (inputs / actions) and postconditions (expected results) which a tester uses to determine whether a system under test satisfies requirements or works correctly. A test case can have one or multiple test scripts and a collection of test cases is called a test suite.

Test Case Template

A test case can have the following elements. Note, however, that a test management tool is normally used by companies and the format is determined by the tool used.

Test Suite ID	The ID of the test suite to which this test case belongs.
Test Case ID	The ID of the test case.
Test Case Summary	The summary / objective of the test case.
Related Requirement	The ID of the requirement this test case relates/traces to.
Prerequisites	Any prerequisites or preconditions that must be fulfilled prior to executing the test.
Test Script / Procedure	Step-by-step procedure to execute the test.
Test Data	The test data, or links to the test data, that are to be used while conducting the test.
Expected Result	The expected result of the test.
Actual Result	The actual result of the test; to be filled after executing the test.
Status	Pass or Fail. Other statuses can be 'Not Executed' if testing is not performed and 'Blocked' if testing is blocked.
Remarks	Any comments on the test case or test execution.
Created By	The name of the author of the test case.
Date of Creation	The date of creation of the test case.
Executed By	The name of the person who executed the test.
Date of Execution	The date of execution of the test.
Test Environment	The environment (Hardware/Software/Network) in which the test was executed.

Test Case Example/ Test Case Sample

Test Suite ID	TS001
Test Case ID	TC001
Test Case Summary	To verify that clicking the Generate Coin button generates coins.
Related Requirement	RS001
Prerequisites	User is authorized. Coin balance is available.
Test Script / Procedure	Select the coin denomination in the Denomination field. Enter the number of coins in the Quantity field. Click Generate Coin.
Test Data	Denominations: 0.05, 0.10, 0.25, 0.50, 1, 2, 5 Quantities: 0, 1, 5, 10, 20
Expected Result	Coin of the specified denomination should be produced if the specified Quantity is valid (1, 5) A message 'Please enter a valid quantity between 1 and 10' should be displayed if the specified quantity is invalid.
Actual Result	If the specified quantity is valid, the result is as expected. If the specified quantity is invalid, nothing happens; the expected message is not displayed
Status	Fail
Remarks	This is a sample test case.
Created By	John Doe
Date of Creation	01/14/2020
Executed By	Jane Roe
Date of Execution	02/16/2020
Test Environment	- OS: Windows Y - Browser: Chrome N

Writing Good Test Cases

- ❑ As far as possible, write test cases in such a way that you test only one thing at a time. Do not overlap or complicate test cases. Attempt to make your test cases 'atomic'.
- ❑ Ensure that all positive scenarios AND negative scenarios are covered.
- ❑ **Language:**
 - Write in simple and easy-to-understand language.
 - Use active voice instead of passive voice: Do this, do that.
 - Use exact and consistent names (of forms, fields, etc).
- ❑ **Characteristics of a good test case:**
 - Accurate:* Exacts the purpose.
 - Economical:* No unnecessary steps or words.
 - Traceable:* Capable of being traced to requirements.
 - Repeatable:* Can be used to perform the test over and over.
 - Reusable:* Can be reused if necessary.

Test Suite

A **TEST SUITE** is a collection of test cases. In automated testing, it can mean a collection of test scripts. In a test suite, the test cases / scripts are organized in a logical order. For example, the test case for registration will precede the test case for login.

Sample Test Cases

Let's assume that you are testing the login functionality of any web application, say Facebook.

Below are the Test Cases for the same:

Test Scenario ID	Login-1	Test Case ID	Login-1A				
Test Case Description	Login – Positive test case	Test Priority	High				
Pre-Requisite	A valid user account	Post-Requisite	NA				
Test Execution Steps:							
S.No	Action	Inputs	Expected Output	Actual Output	Test Browser	Test Result	Test Comments
1	Launch application	https://www.facebook.com/	Facebook home	Facebook home	IE-11	Pass	[Priya 10/17/2017 11:44 AM]: Launch successful
2	Enter correct Email & Password and hit login button	Email id : test@xyz.com Password: *****	Login success	Login success	IE-11	Pass	[Priya 10/17/2017 11:45 AM]: Login successful

Sample Test Cases

Test Scenario ID	Login-1		Test Case ID	Login-1B			
Test Case Description	Login – Negative test case		Test Priority	High			
Pre-Requisite	NA		Post-Requisite	NA			
Test Execution Steps:							
S.No	Action	Inputs	Expected Output	Actual Output	Test Browser	Test Result	Test Comments
1	Launch application	https://www.facebook.com/	Facebook home	Facebook home	IE -11	Pass	[Priya 10/17/2017 11:44 AM]: Launch successful
2	Enter invalid Email & any Password and hit login button	Email id : invalid@xyz.com Password: *****	The email address or phone number that you've entered doesn't match any account. Sign up for an account.	The email address or phone number that you've entered doesn't match any account. Sign up for an account.	IE -11	Pass	[Priya 10/17/2017 11:45 AM]: Invalid login attempt stopped
3	Enter valid Email & incorrect Password and hit login button	Email id : valid@xyz.com Password: *****	The password that you've entered is incorrect. Forgotten password?	The password that you've entered is incorrect. Forgotten password ?	IE -11	Pass	[Priya 10/17/2017 11:46 AM]: Invalid login attempt stopped

Test Execution

Test Execution is an extension of Software Testing Life Cycle. Test execution is the most important part of Software Testing Life Cycle (STLC). Likewise of the entire software development.

In other words, it is the process of executing the code and comparing the expected and actual results.

Factors For A Test Execution Process:

The test execution process need to consider several factors to in order to be effective:

- Select a test suite subset
- Assign the test cases in each test suite to testers for execution.
- Execute tests, report bugs, and capture test status continuously.
- Resolve blocking issues as they arise.
- Report status, adjust assignments, and reconsider plans and priorities daily.
- Report test cycle findings and status.

Equivalence Partitioning & Boundary Value Analysis

Practically, due to time and budget considerations, it is not possible to perform exhausting testing for each set of test data, especially when there is a large pool of input combinations.

- We need an easy way or special techniques that can select test cases intelligently from the pool of test-case, such that all test scenarios are covered.
- We use two techniques - **Equivalence Partitioning & Boundary Value Analysis testing techniques** to achieve this.

Equivalence Partitioning

Equivalence Partitioning or Equivalence Class Partitioning is type of black box testing technique which can be applied to all levels of software testing like unit, integration, system, etc. In this technique, input data units are divided into equivalent partitions that can be used to derive test cases which reduces time required for testing because of small number of test cases.

- It divides the input data of software into different equivalence data classes.
- You can apply this technique, where there is a range in the input field.

Equivalence Partitioning & Boundary Value Analysis

What is Boundary Testing?

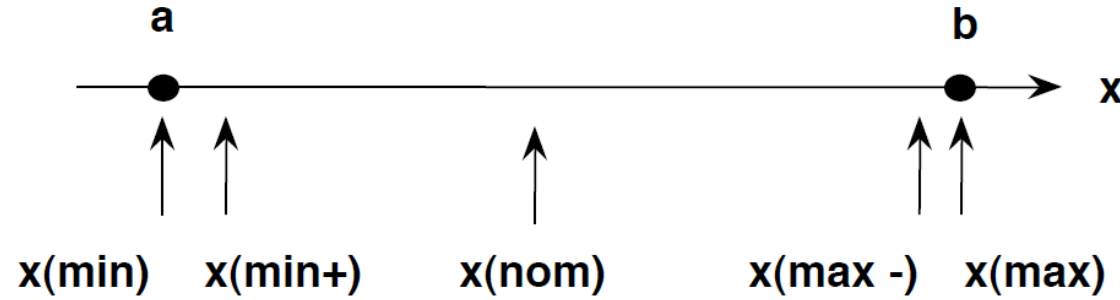
Boundary testing is the process of testing between extreme ends or boundaries between partitions of the input values.

So these extreme ends like Start- End, Lower- Upper, Maximum-Minimum, Just Inside-Just Outside values are called boundary values and the testing is called "boundary testing".

The basic idea in boundary value testing is to select input variable values at their:

1. Minimum
2. Just above the minimum
3. A nominal value
4. Just below the maximum
5. Maximum

Equivalence Partitioning & Boundary Value Analysis



- In Boundary Testing, Equivalence Class Partitioning plays a good role
- Boundary Testing comes after the Equivalence Class Partitioning.

Equivalence Partitioning & Boundary Value Analysis

Example 1: Equivalence and Boundary Value

- Let's consider the behavior of Order Pizza Text Box Below
- Pizza values 1 to 10 is considered valid. A success message is shown.
- While value 11 to 99 are considered invalid for order and an error message will appear, **"Only 10 Pizza can be ordered"**

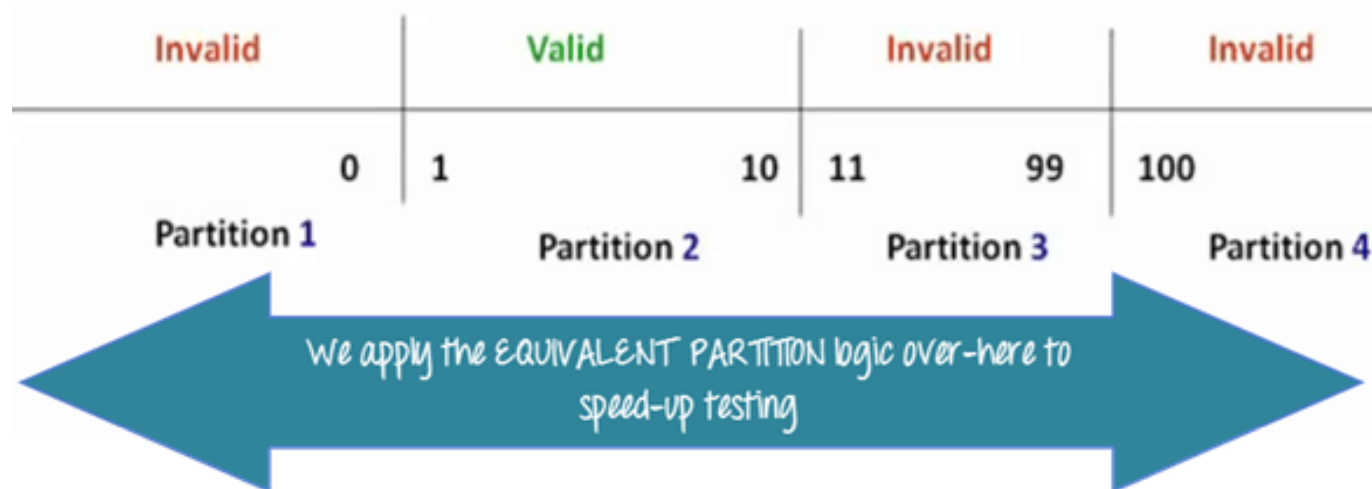
Order Pizza:

Here is the test condition

1. Any Number greater than 10 entered in the Order Pizza field(let say 11) is considered invalid.
2. Any Number less than 1 that is 0 or below, then it is considered invalid.
3. Numbers 1 to 10 are considered valid
4. Any 3 Digit Number say -100 is invalid.

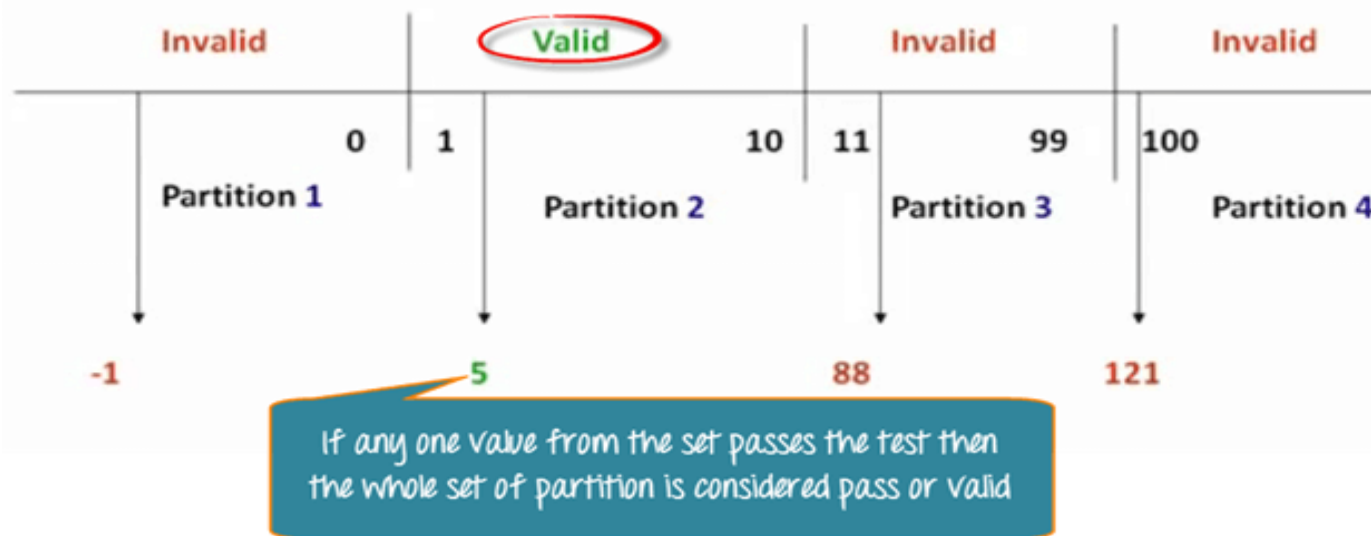
Equivalence Partitioning & Boundary Value Analysis

We cannot test all the possible values because if done, the number of test cases will be more than 100. To address this problem, we use equivalence partitioning hypothesis where we divide the possible values of tickets into groups or sets as shown below where the system behavior can be considered the same.



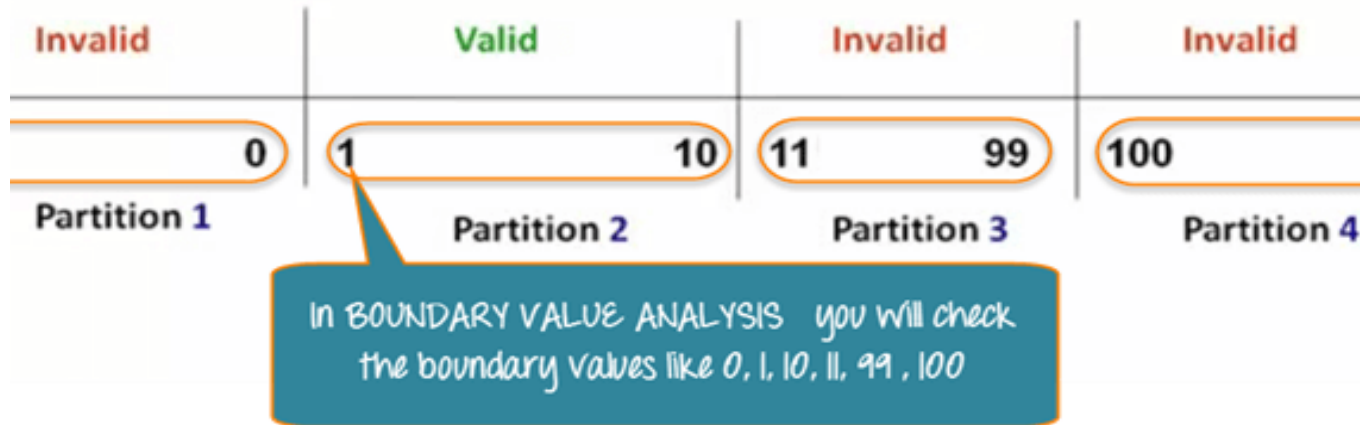
Equivalence Partitioning & Boundary Value Analysis

The divided sets are called Equivalence Partitions or Equivalence Classes. Then we pick only one value from each partition for testing. The hypothesis behind this technique is **that if one condition/value in a partition passes all others will also pass**. Likewise, if one condition in a partition fails, all other conditions in that partition will fail.



Equivalence Partitioning & Boundary Value Analysis

Boundary Value Analysis- in Boundary Value Analysis, you test boundaries between equivalence partitions



In our earlier example instead of checking, one value for each partition you will check the values at the partitions like 0, 1, 10, 11 and so on. As you may observe, you test values at **both valid and invalid boundaries**. Boundary Value Analysis is also called **range checking**.

Equivalence partitioning and boundary value analysis(BVA) are closely related and can be used together at all levels of testing.

Equivalence Partitioning & Boundary Value Analysis

Example 2: Equivalence and Boundary Value

Following password field accepts minimum 6 characters and maximum 10 characters

That means results for values in partitions 0-5, 6-10, 11-14 should be equivalent

Enter Password:

Test Scenario #	Test Scenario Description	Expected Outcome
1	Enter 0 to 5 characters in password field	System should not accept
2	Enter 6 to 10 characters in password field	System should accept
3	Enter 11 to 14 character in password field	System should not accept

Equivalence Partitioning & Boundary Value Analysis

Examples 3: Input Box should accept the Number 1 to 10

Here we will see the Boundary Value Test Cases

Test Scenario Description	Expected Outcome
---------------------------	------------------

Boundary Value = 0	System should NOT accept
--------------------	--------------------------

Boundary Value = 1	System should accept
--------------------	----------------------

Boundary Value = 2	System should accept
--------------------	----------------------

Boundary Value = 9	System should accept
--------------------	----------------------

Boundary Value = 10	System should accept
---------------------	----------------------

Boundary Value = 11	System should NOT accept
---------------------	--------------------------

Equivalence Partitioning & Boundary Value Analysis

Why Equivalence & Boundary Analysis Testing

1. This testing is used to reduce a very large number of test cases to manageable chunks.
2. Very clear guidelines on determining test cases without compromising on the effectiveness of testing.
3. Appropriate for calculation-intensive applications with a large number of variables/inputs

Summary:

- Boundary Analysis testing is used when practically it is impossible to test a large pool of test cases individually
- Two techniques - Equivalence Partitioning & Boundary Value Analysis testing techniques are used
- In Equivalence Partitioning, first, you divide a set of test condition into a partition that can be considered.
- In Boundary Value Analysis you then test boundaries between equivalence partitions
- Appropriate for calculation-intensive applications with variables that represent physical quantities